Docker - Comprehensive Study Notes

Complete Reference | Beginner to Intermediate

Coverage: Why Docker | Images | Containers | VM vs Docker | Commands | Ports | Volumes | Bind Mounts | Dockerfile | Docker Compose | Best Practices

1. Why Docker? - The Problems It Solves

What is it?	Docker is an open-source containerization platform that packages an application with ALL its dependencies into a portable, self-sufficient unit called a container.
Why is it needed?	Before Docker, apps worked on one machine but failed on another due to different dependency versions, OS configs, or missing libraries - causing delays and production failures.
What is the purpose?	Eliminate environment inconsistencies. Make applications run identically on any machine - developer laptop, tester's system, or production server.
Use Case	A developer builds an app on Java 21. The tester has Java 8. Without Docker, the app fails on the tester's machine. With Docker, Java 21 is bundled inside the container - it runs everywhere.

Problem 1: Works on My Machine

- Developer builds app on Java 21 - works perfectly on their system
- Tester has Java 8 - build fails immediately
- Production server has different OS/version - app breaks in production
- Root cause: Different dependency versions across environments
- Real apps have 10+ dependencies - each one can be mismatched

Problem 2: Manual System Setup

- Every new machine needs manual installation of each dependency

- Easy to install the wrong version - app breaks again
- Time-consuming and error-prone process
- Tester's laptop dies -> new laptop -> re-install everything from scratch

Docker's Solution

Docker bundles the application + ALL its dependencies into a single container. The container runs identically on ANY machine that has Docker installed.

- Tester gets one command to start the app - zero manual setup needed
- Same container runs on dev, test, staging, and production
- Permanently eliminates the 'it works on my machine' problem
- Docker is FREE to install and use locally for learning

2. Docker Image

What is it?	A Docker Image is a read-only, immutable template containing the application code, runtime, libraries, environment variables, and all configuration needed to run the application.
Why is it needed?	Containers need a blueprint to know what to include. The image is that blueprint - define it once and create unlimited containers from it.
What is the purpose?	Serve as a portable, shareable, versioned package that guarantees the same environment every single time a container is created from it.
Use Case	Share your app image on DockerHub. Anyone on the team pulls the image and runs an identical container on their machine without installing anything manually.

Key Characteristics

- Read-only - you cannot modify a running image; changes happen inside containers
- Layered - built in layers (each Dockerfile instruction = one cached layer)
- Reusable - one image can create unlimited containers
- Shareable - push to DockerHub for others to pull and run
- Versioned - use tags to maintain multiple versions (1.0.0, 2.0.0, latest)

Image vs Container Analogy

An Image is like a master document (original). A Container is like a photocopy (xerox). You use the original to create as many copies as needed. The original is never changed.

Practical: Build and Check an Image

```
# Build image from Dockerfile in current directory
docker build -t calculator:1.0.0 .

# List all images
docker images
# Output columns: REPOSITORY  TAG    IMAGE ID       CREATED    SIZE
# calculator  1.0.0  a1b2c3d4e5f6   2 mins ago   1.1GB

# Inspect image details (JSON metadata)
docker inspect calculator:1.0.0

# View build layers
docker history calculator:1.0.0
```

3. Docker Container

What is it?	A Docker Container is a running instance of a Docker Image. It is a lightweight, isolated process that runs the application in its own filesystem, network, and process space.
Why is it needed?	Images are static blueprints. We need an actual RUNNING environment for the application. Containers are that live execution environment.
What is the purpose?	Provide an isolated, consistent, and portable runtime environment for any application without needing a full virtual machine.
Use Case	Run 10+ microservices simultaneously on one machine, each in its own container with its own dependencies, no conflicts between them.

Container vs Virtual Machine Comparison

Aspect	Virtual Machine (VM)	Docker Container
OS	Includes a FULL guest OS per VM (Ubuntu = 2 GB)	Shares host OS kernel - no guest OS needed
Disk Size	2-4 GB per VM (OS + app)	~400 MB (app + dependencies only)
RAM Usage	Needs dedicated RAM for guest OS (2-4 GB extra)	Uses only what the app actually needs
Boot Time	10-20 minutes (full OS boot like Windows startup)	Seconds (no OS to boot - just starts the process)
Density	~2 VMs on an 8 GB machine	10+ containers on the same 8 GB machine
Isolation	Full hardware-level isolation	Process-level isolation (lighter but sufficient)

Why Docker Won Over VMs

- VMs carry an entire OS per application - massive overhead
- Docker reuses the host OS kernel - no OS overhead per container
- A 200 MB app in a container = ~400 MB total; same app in a VM = 2-4 GB
- VMs take 10-20 minutes to boot; containers start in seconds
- This efficiency enabled Kubernetes, Jenkins, and modern CI/CD

4. Docker Engine (Daemon)

What is it?	Docker Engine is the core background service (daemon) that builds images, creates/manages containers, handles networks, and manages volumes. The daemon process is called 'dockerd'.
Why is it needed?	Someone needs to do the actual work of building images and running containers. The Docker Engine is that runtime process running in the background.
What is the purpose?	Act as the server-side component handling all Docker operations. It listens for Docker CLI commands via the Docker API and executes them.
Use Case	When you type 'docker build' or 'docker run', the Docker CLI sends the command to the Docker Engine daemon which actually does the work.

Architecture

- Docker Daemon (dockerd) - background service that manages all Docker objects
- Docker CLI - command-line tool you use; sends commands to the daemon
- Docker API - REST API the daemon exposes; the CLI uses it internally
- Docker Desktop - GUI application wrapping the engine for Mac/Windows

Check Docker is running: docker --version OR docker info (if command fails, open Docker Desktop first)

Docker CLI vs Docker Desktop

- CLI (terminal/command prompt) is the primary way professionals use Docker
- Docker Desktop provides a GUI to view containers, images, and volumes
- Everything you do on Docker Desktop can also be done via CLI
- In production servers there is NO GUI - only CLI commands
- CLI commands - they work everywhere; GUI is just for convenience

5. Docker Commands - Complete Reference

Version & Info

Command	Description
`docker --version`	Show Docker version
`docker version`	Detailed client + server version info
`docker info`	System info: containers count, images, storage driver

Image Commands

Command	Description
`docker images`	List all local images
`docker images -a`	List all images including intermediate layers
`docker pull <image>:<tag>`	Download image from DockerHub
`docker build -t <name>:<tag> .`	Build image from Dockerfile in current directory
`docker build -t <name> -f /path/Dockerfile .`	Build using Dockerfile at specific path
`docker tag <src> <repo/name:tag>`	Tag image for pushing to a registry
`docker push <repo/name:tag>`	Upload/push image to DockerHub
`docker rmi <image>`	Remove/delete a local image
`docker rmi $(docker images -q)`	Delete ALL local images
`docker image prune`	Remove all unused/dangling images
`docker history <image>`	Show layer history of an image
`docker inspect <image>`	Show detailed JSON metadata of an image
`docker save -o file.tar <image>`	Save image to a tar archive file
`docker load -i file.tar`	Load image from a tar archive file

Container Commands

Command	Description
`docker run <image>`	Create and start a container
`docker run -d <image>`	Run container in detached (background) mode
`docker run -p 8080:3000 <image>`	Map host port 8080 to container port 3000
`docker run --name myapp <image>`	Assign a custom name to the container
`docker run -e KEY=VALUE <image>`	Set environment variable inside container
`docker run --rm <image>`	Auto-remove container after it stops
`docker run -it <image> bash`	Run container interactively with bash shell
`docker run -v vol:/path <image>`	Mount a named volume
`docker run -v $(pwd)/src:/app <image>`	Bind mount host folder to container
`docker ps`	List RUNNING containers only
`docker ps -a`	List ALL containers (running + stopped)
`docker start <container>`	Start a stopped container

`docker stop <container>`	Gracefully stop a running container (SIGTERM)
`docker kill <container>`	Force stop a container immediately (SIGKILL)
`docker restart <container>`	Stop and restart a container
`docker rm <container>`	Remove a stopped container
`docker rm -f <container>`	Force remove a running container
`docker rm $(docker ps -aq)`	Remove ALL stopped containers
`docker logs <container>`	View logs from a container
`docker logs -f <container>`	Follow/stream logs in real-time
`docker logs --tail 100 <container>`	Show last 100 lines of logs
`docker exec -it <container> bash`	Open bash shell inside running container
`docker exec <container> <command>`	Run any command inside running container
`docker inspect <container>`	Detailed JSON info about a container
`docker stats`	Live CPU/memory usage of all containers
`docker stats <container>`	Live stats for specific container
`docker top <container>`	Show processes running inside container
`docker cp <container>:/path ./local`	Copy file FROM container to host
`docker cp ./local <container>:/path`	Copy file FROM host to container

Registry Commands

Command	Description
`docker login`	Login to DockerHub (interactive)
`docker login -u <user> -p <pass>`	Login non-interactively
`docker logout`	Logout from registry
`docker search <term>`	Search DockerHub for images

Cleanup Commands

Command	Description
`docker system prune`	Remove stopped containers + unused networks + dangling images
`docker system prune -a`	Remove all unused images too
`docker system df`	Show disk usage by Docker objects
`docker container prune`	Remove all stopped containers
`docker image prune -a`	Remove all unused images
`docker volume prune`	Remove all unused volumes
`docker network prune`	Remove all unused networks

6. DockerHub - Image Registry

What is it?	DockerHub is the default cloud-based container image registry at hub.docker.com. It stores and distributes Docker images publicly or privately - like GitHub for code, but for Docker images.
Why is it needed?	We need a way to share images between team members, CI/CD pipelines, and production servers. Email or file transfer is impractical for GB-sized images.
What is the purpose?	Centralized storage for Docker images enabling easy distribution, versioning, and collaboration across teams and deployment environments.
Use Case	Developer builds image -> pushes to DockerHub -> Tester pulls the exact same image and runs it -> CI/CD pipeline also pulls it for automated testing -> same image deployed to production.

Push Workflow (Share an Image)

```
# Step 1: Login to DockerHub
docker login

# Step 2: Tag image with your DockerHub username/repo
# Format: docker tag <local_image> <username>/<repo>:<version>
docker tag calculator xrepo/calculator:1.0.0

# Step 3: Push to DockerHub
docker push xrepo/calculator:1.0.0
# Image is now available at: hub.docker.com/r/xrepo/calculator
```

Pull Workflow (Download an Image)

```
# Pull image from DockerHub
docker pull xrepo/calculator:1.0.0

# Run it
docker run -p 9000:5000 xrepo/calculator:1.0.0

# Note: 'docker run' auto-pulls if image not found locally
docker run -p 9000:5000 xrepo/calculator:1.0.0
# If not local -> pulls automatically -> runs
```

Other Registries

- AWS ECR - Amazon Elastic Container Registry
- Google Artifact Registry (GCR) - Google Container Registry
- Azure ACR - Azure Container Registry
- GitHub GHCR - GitHub Container Registry (ghcr.io)
- Self-hosted - Docker Registry or Harbor for private on-premises use

7. Image Versioning with Tags

What is it?	A Docker tag is a label attached to an image identifying a specific version. Format: imagename:tag (e.g. calculator:1.0.0, node:20, ubuntu:22.04).
Why is it needed?	Without versioning, updating an image overwrites the previous version. If the new version breaks production, there is no way to roll back quickly.
What is the purpose?	Maintain multiple stable versions simultaneously. Enable instant rollback to a known-good version if a new deployment fails.
Use Case	v1.0.0 runs in production. You deploy v2.0.0 which has a bug. You roll back to v1.0.0 with one command - zero downtime, immediate recovery.

Tag Commands

```
# Tag at build time
docker build -t calculator:1.0.0 .
docker build -t calculator:2.0.0 .

# Tag existing image (add new tag to existing image)
docker tag calculator xrepo/calculator:1.0.0

# 'latest' is applied automatically if no tag is given
docker build -t calculator .     # tagged as calculator:latest

# Pull a specific version
docker pull node:20
docker pull node:18-alpine
docker pull ubuntu:22.04
```

Versioning Best Practices

- NEVER use 'latest' in production - it changes unpredictably
- Use Semantic Versioning: MAJOR.MINOR.PATCH (e.g. 1.0.0, 1.1.0, 1.1.1)
- Tag with git commit hash for traceability: calculator:abc1234
- Keep old versions available in registry for rollback capability
- Use Alpine variants (node:20-alpine) for smaller, faster production images
- Always pull latest base images periodically to get security patches

8. Port Mapping (Port Binding)

What is it?	Port mapping (-p) connects a port on the HOST machine to a port inside the Docker container. This bridges external traffic into the isolated container network.
Why is it needed?	Containers are fully isolated. Traffic to your machine does not automatically reach inside the container. Port mapping creates a specific bridge for communication.
What is the purpose?	Allow browsers, API clients, and other services to reach the application running inside a container through a specific host port.
Use Case	App runs on port 5000 inside the container. Map host port 9002 to container port 5000. Visit http://localhost:9002 in browser - traffic is forwarded to port 5000 inside container.

Port Mapping Syntax

```
# Syntax
docker run -p <HOST_PORT>:<CONTAINER_PORT> <image>

# Example: host port 9002 -> container port 5000
docker run -p 9002:5000 calculator

# Multiple port mappings
docker run -p 80:8080 -p 443:8443 myapp

# Map to all network interfaces (default behavior)
docker run -p 0.0.0.0:8080:8080 myapp

# Map to localhost only (more secure)
docker run -p 127.0.0.1:8080:8080 myapp

# Random host port (Docker chooses free port)
docker run -p :5000 myapp
docker port myapp   # see which port was assigned
```

CRITICAL RULE: The CONTAINER port (right side of colon) must EXACTLY match the port your app listens on inside the container. If your app runs on port 5000, you must write :5000. Using :8081 when app is on :5000 = no response.

Port Concepts

- One port can only host ONE application at a time (like one train per railway track)
- Each application needs a unique port (0-65535 range)
- Host port can be any free port on your machine
- Container port must match your app's configured listen port
- Common ports: 80 (HTTP), 443 (HTTPS), 3000 (React/Node), 8080 (Spring/Java), 5432 (PostgreSQL), 27017 (MongoDB), 6379 (Redis)

9. Docker Volumes

What is it?	A Docker Volume is persistent storage managed by Docker. Data written to a volume survives even if the container is stopped, crashed, or deleted.
Why is it needed?	Containers are ephemeral - when deleted, ALL data inside is permanently lost. Databases and stateful applications need data that outlives container restarts.
What is the purpose?	Decouple data lifecycle from container lifecycle. Mount the same volume to a new container and recover all previous data instantly.
Use Case	Calculator app stores user history inside the container. Container crashes -> history is gone. With a volume, history is saved to the volume -> mount same volume to new container -> history restored.

Volume Commands

```
# Create a named volume
docker volume create my-calculator-volume

# List all volumes
docker volume ls

# Inspect volume (shows where files are stored on host)
docker volume inspect my-calculator-volume
# Shows: Mountpoint -> /var/lib/docker/volumes/my-calculator-volume/_data

# Run container WITH volume mounted
# Syntax: -v <volume_name>:/path/inside/container
docker run -p 9000:5000 -v my-calculator-volume:/data calculator

# Mount volume with new container (data is still there!)
docker run -p 9001:5000 -v my-calculator-volume:/data calculator

# Remove a specific volume
docker volume rm my-calculator-volume

# Remove all unused volumes
docker volume prune
```

How Volumes Work

- Volume files are stored in a Docker-managed location on the host: /var/lib/docker/volumes/
- Docker manages this folder - you cannot easily browse it directly
- Container writes to /data inside -> actually saved to volume on host
- Delete container -> volume and ALL its data remain intact
- Mount the same volume to a new container -> old data available immediately
- Volumes must be mounted at container creation time (docker run), NOT after
- Multiple containers can share the same volume (read/write access)

10. Bind Mounts

What is it?	A Bind Mount links a specific folder on the HOST machine directly to a folder inside the container. Both the host and container see the exact same files in real-time.
Why is it needed?	Docker Volumes store files in a hidden Docker-managed location you cannot access directly. Bind Mounts store files in YOUR chosen folder - fully accessible from your OS.
What is the purpose?	Give you direct read/write access to container data from the host, and allow live injection of files into the container without rebuilding the image.
Use Case	Development: mount your source code folder into container so code changes appear immediately without rebuilding. Production: mount config files, SSL certificates, or log directories.

Volume Mount vs Bind Mount

Aspect	Volume Mount	Bind Mount
Storage location	Docker-managed (/var/lib/docker/volumes/)	Any path you choose on the host
Host access	Cannot access files directly from host	Full read/write access from host machine
Portability	Works same on any Docker host	Path must exist on destination host
Best for	Production databases, persistent app data	Development, config files, certificates

Bind Mount Command

```
# Syntax (use absolute path for host folder)
docker run -v /absolute/host/path:/container/path <image>

# Using $(pwd) for current directory
docker run -p 9000:5000 -v $(pwd)/data:/data calculator

# Development: live source code mounting
docker run -p 3000:3000 -v $(pwd)/src:/app/src myapp

# Mount a single config file
docker run -v $(pwd)/nginx.conf:/etc/nginx/nginx.conf nginx

# Read-only bind mount (container can read but not modify)
docker run -v $(pwd)/config:/app/config:ro myapp
```

Key Advantages of Bind Mounts

- Add a file to your host folder -> instantly appears inside the container
- Modify code in VS Code -> container sees the change immediately (hot-reload)
- Access/export container data files directly from your host file system
- Share files easily with your manager - just open the linked host folder
- Essential for development workflows; not recommended for production

11. Dockerfile

What is it?	A Dockerfile is a plain text file named 'Dockerfile' (capital D, rest lowercase) containing instructions Docker uses to automatically build a Docker image layer by layer.
Why is it needed?	Docker needs to know what goes into the image: which base OS/runtime, which dependencies to install, what code to include, and how to start the application.
What is the purpose?	Automate image creation with a reproducible, version-controlled, shareable recipe. One Dockerfile produces an identical image on any machine.
Use Case	Instead of manually installing Node 20, copying files, running npm install on each machine - write a Dockerfile once and 'docker build' handles everything automatically.

Naming Rules

- File name must be exactly: Dockerfile (capital D, rest lowercase)
- Case-sensitive: 'dockerfile' or 'DOCKERFILE' will not work by default
- No file extension - not Dockerfile.txt, just Dockerfile
- Place it in the root of your project folder

Dockerfile Instructions

Command	Description
`FROM <image>:<tag>`	Set base image (REQUIRED, must be first instruction)
`WORKDIR /path`	Set working directory for all subsequent instructions
`COPY <src> <dest>`	Copy files/folders from host build context into image
`ADD <src> <dest>`	Like COPY but also unpacks .tar files, supports URLs
`RUN <command>`	Execute shell command at BUILD time (install packages etc)
`ENV KEY=VALUE`	Set environment variable available at build and runtime
`ARG NAME=default`	Build-time variable only (not available at runtime)
`EXPOSE <port>`	Document which port app listens on (informational only)
`CMD ["cmd","arg"]`	Default command when container starts (overridable)
`ENTRYPOINT ["cmd"]`	Fixed startup command (CMD provides default arguments)
`VOLUME ["/data"]`	Declare a mount point for a persistent volume
`USER <username>`	Switch to non-root user for security
`LABEL key=value`	Add metadata labels to the image

EXPOSE vs Port Mapping

- EXPOSE in Dockerfile = documentation only. Tells developers which port the app uses.
- EXPOSE does NOT actually publish the port or make it accessible from outside
- -p flag in 'docker run' = actually maps/publishes the port
- Even without EXPOSE in Dockerfile, you can use -p to map ports

CMD vs ENTRYPOINT

- CMD - default command; OVERRIDABLE when running container
- ENTRYPOINT - fixed command; arguments are APPENDED to it
- Use CMD when container might run different commands
- Use ENTRYPOINT when container IS a specific tool (e.g. a script runner)

```
# CMD - can be overridden
CMD ["node", "server.js"]
docker run myimage python app.py  # overrides CMD entirely

# ENTRYPOINT + CMD (best practice)
ENTRYPOINT ["node"]
CMD ["server.js"]              # default argument
docker run myimage app.js      # runs: node app.js
```

Complete Dockerfile Example - Node.js App

```
# Stage: Production-Base image
FROM node:20

# Set working directory (created automatically if missing)
WORKDIR /calculator

# Copy package files FIRST (cache optimization)
# If package.json unchanged, npm install layer is cached
COPY package.json package-lock.json ./

# Install dependencies
RUN npm install

# Copy remaining source files
COPY . .

# Document the port (informational)
EXPOSE 5000

# Run as non-root for security
USER node

# Start command (runs when container starts)
CMD ["node", "server.js"]
```

Base Image Strategy

You don't need to build everything from scratch. Official images on DockerHub have the runtime pre-installed.

- FROM ubuntu:22.04 -> bare Ubuntu; install everything yourself
- FROM node:20 -> Ubuntu + Node.js 20 already installed (saves time)
- FROM node:20-alpine -> Alpine Linux + Node.js (~50 MB vs ~1 GB) - much leaner
- FROM python:3.11-slim -> slim Python image

- Find all official images at: hub.docker.com

.dockerignore File

Exclude files from the build context. Speeds up builds and prevents secrets from leaking into images.

```
# .dockerignore  (place next to Dockerfile)
node_modules
.git
.env
*.log
dist
coverage
.DS_Store
Dockerfile
README.md
```

12. Practical Guide - Start to Finish

Step 1: Verify Docker is Running

```
docker --version
# Docker version 26.x.x, build abc123

docker info
# System-wide info: containers, images, storage driver
```

Step 2: Write Application (server.js)

```
const http = require('http');
const fs   = require('fs');

const server = http.createServer((req, res) => {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.end(fs.readFileSync('./calculator.html'));
});

server.listen(5000, () => {
  console.log('Calculator running on port 5000');
});
```

Step 3: Create Dockerfile

```
# Project structure:
# myproject/
#   Dockerfile
#   package.json
#   server.js
#   calculator.html
#   .dockerignore

FROM node:20
WORKDIR /calculator
COPY package.json .
RUN npm install
COPY . .
EXPOSE 5000
CMD ["node", "server.js"]
```

Step 4: Build the Image

```
# -t = tag/name, . = Dockerfile is in current directory
docker build -t calculator:1.0.0 .

# Watch the build output - each step = one layer
# Step 1/6 : FROM node:20  -> downloads base image
# Step 2/6 : WORKDIR /calculator
# Step 3/6 : COPY package.json .
# Step 4/6 : RUN npm install
```

```
# Step 5/6 : COPY . .
# Step 6/6 : CMD ["node", "server.js"]
# Successfully built abc123456

# Verify image was created
docker images
# REPOSITORY   TAG     IMAGE ID       CREATED         SIZE
# calculator   1.0.0   abc123456789   2 seconds ago   1.1GB
```

Step 5: Run the Container

```
# -d = detached background mode
# -p 9002:5000 = host port 9002 -> container port 5000
# --name = friendly container name
docker run -d -p 9002:5000 --name my-calculator calculator:1.0.0

# Verify it is running
docker ps
# CONTAINER ID   IMAGE              STATUS    PORTS
# abc123def456   calculator:1.0.0   Up 5s     0.0.0.0:9002->5000/tcp

# Open in browser: http://localhost:9002
```

Step 6: Manage the Container

```
# View logs
docker logs my-calculator
docker logs -f my-calculator    # stream in real-time

# Open shell inside container (for debugging)
docker exec -it my-calculator bash

# Stop gracefully
docker stop my-calculator

# Restart
docker start my-calculator
```

Step 7: Push to DockerHub

```
docker login
docker tag calculator:1.0.0 yourusername/calculator:1.0.0
docker push yourusername/calculator:1.0.0
```

Step 8: Pull and Run on Another Machine

```
docker pull yourusername/calculator:1.0.0
docker run -d -p 9002:5000 yourusername/calculator:1.0.0
# App is running - no Node.js installation required!
```

Step 9: Cleanup

```
docker stop my-calculator          # stop container
docker rm my-calculator            # delete container
docker rmi calculator:1.0.0        # delete image

# Clean everything (containers + images + networks)
docker system prune -a
```

13. Docker Networking

What is it?	Docker networks allow containers to communicate with each other and with the outside world. Each container gets its own isolated network namespace.
Why is it needed?	Containers are isolated by default. For a web app to talk to a database container, they need to be on the same Docker network.
What is the purpose?	Enable secure, isolated communication between containers without exposing internal ports to the host network unnecessarily.
Use Case	Web app container and PostgreSQL container on the same Docker network. Web app connects to 'db:5432' (container name as hostname) - no extra port mapping needed between them.

Network Commands

```
# List all networks
docker network ls
# NETWORK ID   NAME      DRIVER    SCOPE
# abc123       bridge    bridge    local   <- default
# def456       host      host      local
# ghi789       none      null      local

# Create a custom network
docker network create my-app-network

# Run containers on same network
docker run -d --network my-app-network --name db postgres:15
docker run -d --network my-app-network --name webapp myapp

# webapp can connect to db using hostname 'db'
# DATABASE_URL=postgresql://db:5432/mydb

# Inspect network - see which containers are connected
docker network inspect my-app-network

# Remove network
docker network rm my-app-network
```

Network Types

- bridge (default) - containers on same bridge network communicate by container name
- host - container shares host's network stack; fastest but no isolation
- none - no network access at all; fully isolated
- overlay - multi-host networking for Docker Swarm / Kubernetes
- Custom bridge networks are recommended over the default bridge network

14. Docker Compose

What is it?	Docker Compose is a tool for defining and running multi-container applications using a single YAML file (docker-compose.yml). It manages services, networks, and volumes declaratively.
Why is it needed?	Running multiple containers with separate 'docker run' commands is tedious, error-prone, and hard to share with team members. Compose defines everything in one place.
What is the purpose?	Define your entire application stack (frontend + backend + database + cache) in one file. Start the entire stack with one command; stop it with one command.
Use Case	A full-stack app with React frontend, Node.js backend, PostgreSQL database, and Redis cache - all in one docker-compose.yml. 'docker compose up' starts everything.

Example docker-compose.yml

```
version: '3.8'

services:
  webapp:
    build: .                    # build from local Dockerfile
    ports:
      - '9000:5000'
    environment:
      - DATABASE_URL=postgresql://db:5432/mydb
      - NODE_ENV=production
    depends_on:
      - db
    volumes:
      - ./src:/app/src          # bind mount for development

  db:
    image: postgres:15          # use official image
    environment:
      - POSTGRES_DB=mydb
      - POSTGRES_USER=admin
      - POSTGRES_PASSWORD=secret
    volumes:
      - db-data:/var/lib/postgresql/data   # named volume
    ports:
      - '5432:5432'

  redis:
    image: redis:7-alpine
    ports:
      - '6379:6379'

volumes:
  db-data:                      # declare named volume
```

Docker Compose Commands

Command	Description
`docker compose up`	Start all services (build images if needed)
`docker compose up -d`	Start all services in detached (background) mode
`docker compose up --build`	Force rebuild images before starting
`docker compose down`	Stop and remove containers (volumes preserved)
`docker compose down -v`	Stop, remove containers AND volumes
`docker compose ps`	List running services
`docker compose logs -f`	Stream logs from all services
`docker compose logs -f webapp`	Stream logs from specific service
`docker compose build`	Build/rebuild images without starting
`docker compose restart`	Restart all services
`docker compose stop`	Stop services without removing containers
`docker compose exec webapp bash`	Open shell in running service
`docker compose pull`	Pull latest versions of all service images

15. Docker Best Practices

Dockerfile Best Practices

- Use official base images from DockerHub (node, python, openjdk, etc.)
- Pin specific version tags - use node:20.19.2, not node:latest in production
- Use Alpine variants for smaller images: node:20-alpine (~50 MB vs ~1 GB)
- Copy package.json FIRST, install, THEN copy source - maximizes layer cache
- Use .dockerignore to exclude node_modules, .git, .env from build context
- Run as non-root user: USER node (security best practice)
- One process per container (single responsibility principle)
- Combine RUN commands with && to reduce layer count

Layer Caching Optimization

```
# WRONG - copies ALL files first, cache busted on any change
COPY . .
RUN npm install

# CORRECT - install dependencies first (cached if package.json unchanged)
COPY package.json package-lock.json ./
RUN npm install
COPY . .            # only this layer rebuilds when code changes
```

Multi-Stage Build (Smaller Production Images)

```
# Stage 1: Build environment
FROM node:20 AS builder
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
RUN npm run build

# Stage 2: Production (only the built output, no build tools)
FROM node:20-alpine AS production
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/package.json .
RUN npm install --production
USER node
CMD ["node", "dist/server.js"]
# Result: ~200 MB instead of ~1.1 GB
```

Security Best Practices

- Never store passwords/API keys in Dockerfiles or images (visible in layers)
- Use environment variables or Docker secrets for sensitive data
- Scan images for vulnerabilities: docker scout cves <image>
- Keep base images updated regularly for security patches

- Use USER instruction to run as non-root inside container
- Set read-only root filesystem where possible
- Use --no-new-privileges flag when running containers

Production Best Practices

- Always use specific version tags - never :latest in production
- Set resource limits: docker run --memory=512m --cpus=0.5 myapp
- Configure health checks in Dockerfile or docker-compose.yml
- Store stateful data in volumes - never inside containers
- Use Docker Compose for local dev; Kubernetes for production orchestration
- Implement proper logging with structured log format
- Always test images locally before pushing to registry

16. Quick Reference Cheat Sheet

Core Concepts at a Glance

Term	What It Is	Analogy
Image	Read-only template/blueprint for containers	Master copy / Original document
Container	Running instance of an image	Photocopy / Xerox from original
Dockerfile	Instructions to build an image	Recipe for cooking a dish
Volume	Persistent storage that outlives containers	External hard drive for container
Bind Mount	Link host folder to container folder	Shared folder between host and container
DockerHub	Cloud registry for storing/sharing images	GitHub but for Docker images
Docker Compose	Multi-container app definition in one YAML file	Orchestra conductor for containers
Port Mapping	Connect host port to container port	Doorbell routed to specific room

Most Used Commands

```
# Build
docker build -t myapp:1.0 .              # build image

# Run
docker run -d -p 8080:3000 myapp:1.0    # run in background
docker run -it myapp bash                # run interactively

# Inspect
docker images                            # list images
docker ps                                # list running containers
docker ps -a                             # all containers
docker logs -f <container>              # stream logs
docker exec -it <container> bash        # shell into container
docker stats                             # live resource usage

# Manage
docker stop <container>                 # stop
docker start <container>                # start stopped container
docker rm <container>                   # delete container
docker rmi <image>                      # delete image

# Registry
docker pull <image>:<tag>               # download
docker push <repo/image:tag>            # upload
docker login                            # authenticate

# Volumes
docker volume create mydata            # create volume
docker run -v mydata:/app/data img   # mount volume
docker run -v $(pwd)/src:/app img    # bind mount

# Cleanup
docker system prune -a                 # clean everything unused
```

Dockerfile Quick Reference

```
FROM node:20                 # base image (required first)
WORKDIR /app                 # working directory
COPY package.json .          # copy file
RUN npm install              # run build command
COPY . .                     # copy all source
ENV PORT=3000                # set env var
EXPOSE 3000                  # document port
USER node                    # non-root user
CMD ["node", "server.js"]    # startup command
```